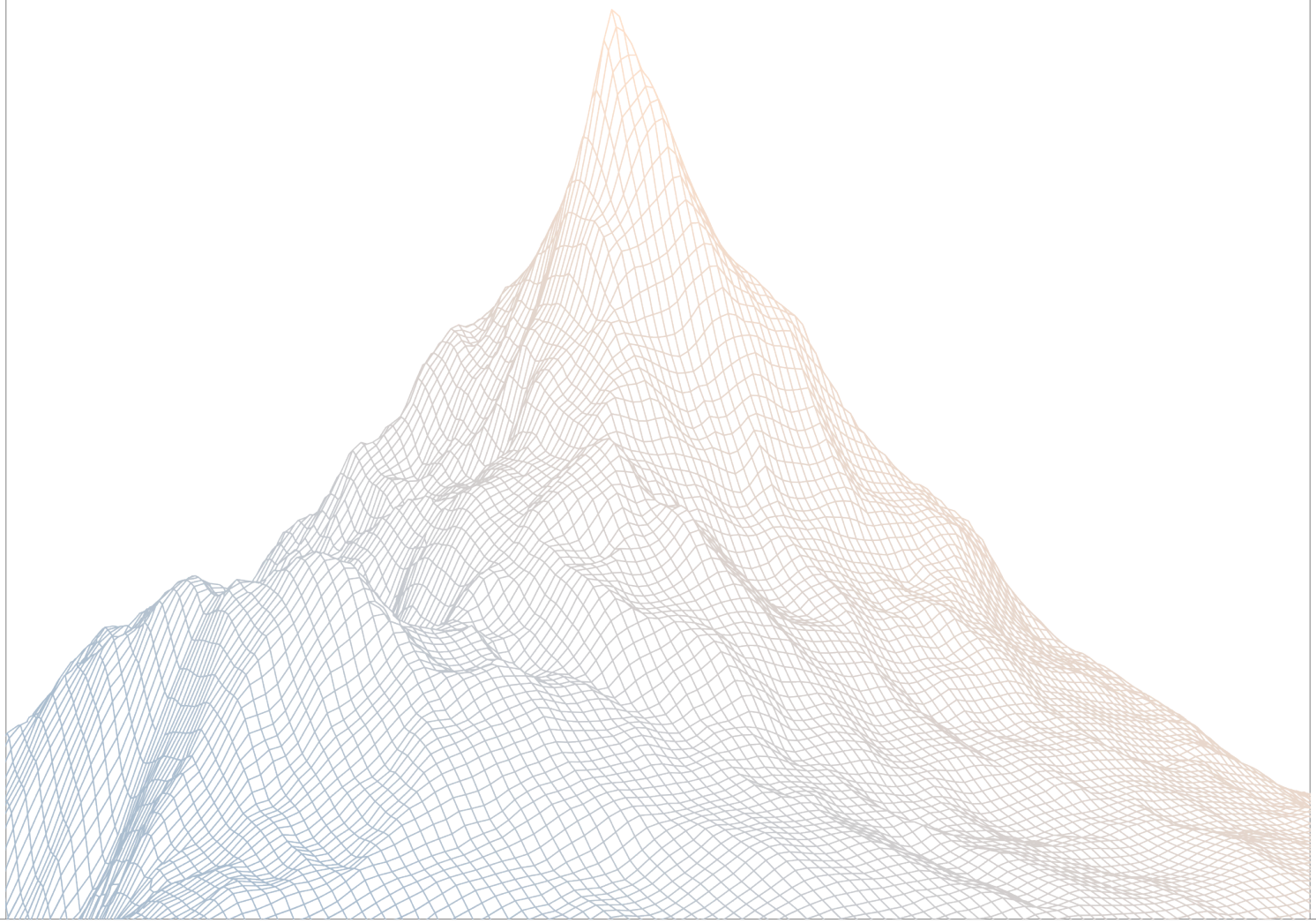
Zenith

Treasures

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: November 7th to November 11th, 2025

AUDITED BY: cccz
rscodes

Contents

1 Introduction 2

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 3

2.1 About Treasures 4

2.2 Scope 4

2.3 Audit Timeline 5

2.4 Issues Found 5

3 Findings Summary 5

4 Findings 6

4.1 High Risk 7

4.2 Medium Risk 9

4.3 Low Risk 16

4.4 Informational 21

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Treasures

A revolutionary take on RWAs, Strategies and Capital Allocation. Treasures is the launchpad underpinning this new investment category.

2.2 Scope

The engagement involved a review of the following targets:

Target	contracts
Repository	https://github.com/treasures-io/contracts
Commit Hash	`c7e9633b31a1091435937f751372bf4f1258b0e3`
Files	`genesis/Genesis.sol` `fund/FundToken.sol`

2.3 Audit Timeline

November 7, 2025	Audit start
November 11, 2025	Audit end
November 21, 2025	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	1
Medium Risk	3
Low Risk	3
Informational	1
Total Issues	**8**

3

Findings Summary

ID	Description	Status
H-1	autoSwap can lead to losses for liquidity providers	Acknowledged
M-1	Tokens raised can exceed maxContributionAmount by a significant amount	Acknowledged
M-2	projectTaxPendingSwap may exceed type(uint128).max	Resolved
M-3	Insufficient slippage and deadline checks in _swapTax()	Resolved
L-1	End time cannot be adjusted once start time starts	Acknowledged
L-2	lpSupply and contributionTokensForLpPool cannot be 0	Resolved
L-3	The actual swapped tax amount will be 1 less than the swapBalance	Resolved
I-1	cancelGenesis has a redundant modifier	Resolved

4

Findings

4.1 High Risk

A total of 1 high risk findings were identified.

[H-1] autoSwap can lead to losses for liquidity providers

SEVERITY: High	IMPACT: High
STATUS: Acknowledged	LIKELIHOOD: Medium

Target

- FundToken.sol#L648-L655

Description:

When the user adds liquidity, tokens are sent to UniswapV2Pair, which may trigger autoSwap. And the autoSwap affecting the user's liquidity additions.

```
function addLiquidity(
    address tokenA,
    address tokenB,
    uint amountADesired,
    uint amountBDesired,
    uint amountAMin,
    uint amountBMin,
    address to,
    uint deadline
) external virtual override ensure(deadline) returns (uint amountA,
    uint amountB, uint liquidity) {
    (amountA, amountB) = _addLiquidity(tokenA, tokenB, amountADesired,
    amountBDesired, amountAMin, amountBMin);
    address pair = UniswapV2Library.pairFor(factory, tokenA, tokenB);
    TransferHelper.safeTransferFrom(tokenA, msg.sender, pair, amountA);
    TransferHelper.safeTransferFrom(tokenB, msg.sender, pair, amountB);
    liquidity = IUniswapV2Pair(pair).mint(to);
}
...
function _eligibleForSwap(address from_, address to_, uint256 taxBalance_,
    uint256 swapThresholdInTokens_)
    internal
```

```
    view
    returns (bool)
{
    return (taxBalance_ >= swapThresholdInTokens_ && !_autoSwapInProgress &&
    !isLiquidityPool(from_)
            && from_ ≠ address(_uniswapRouter) && to_ ≠
    address(_uniswapRouter) && from_ ≠ address(this));
}
```

Since UniswapV2Pair will mint liquidity on the side of lesser amount, this may cause the liquidity provider to loss.

```
} else {
    liquidity = Math.min(amount0.mul(_totalSupply) / _reserve0,
    amount1.mul(_totalSupply) / _reserve1);
}
```

For example, if the user provides tokens according to pool tokens ratio, but autoSwap is triggered when transferring tokens, then the pool tokens ratio changes and the user loses some tokens.

Recommendations:

One option is to disable autoSwap when `to` is a liquidity pool, but this could affect the swapping process, as the protocol prefers autoSwap before the actual swap to make prices fresh.

Another option is to implement a new Router contract so that when adding liquidity, tokens are first sent to the router, then to the pair. This would also prevent autoSwap.

Treasures: Acknowledged. This is a valid concern, however (1) the protocol (and business model) requires autoswap to work (2) we expect external provision of liquidity to be rare (3) in the event when losses happen due to external provision, we have the option to consider manually refund the liquidity provider the lost tokens from our tax treasury.

4.2 Medium Risk

A total of 3 medium risk findings were identified.

[M-1] Tokens raised can exceed `maxContributionAmount` by a significant amount

SEVERITY: Medium	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Medium

Target

- Genesis.sol

Description:

The genesis contract has a `maxContributionAmount` variable to ensure that the contract does not receive too much contribution tokens past the set amount.

However we can see that the check against the `maxContributionAmount` is done at the start of `contribute()` instead of at the end.

Namely, `whenActive` is called.

```
function contribute(uint256 amount) external nonReentrant whenActive {
    ....
    totalRaised += amount;
    ....
}
```

And `whenActive` checks `isEnded()` which checks `reachedMaxContributionAmount`:

```
function isEnded() public view returns (bool) {
    return reachedEndTime() || reachedMaxContributionAmount();
}
function reachedMaxContributionAmount() public view returns (bool) {
    return totalRaised >= maxContributionAmount;
}
```

Hence consider the following sequence:

1. `maxContributionAmount = 100 ether`;
2. `totalRaised = 98 ether`;
3. `Alice` calls `contribute` with `20 ether`;
4. `totalRaised` becomes `118 ether` at the end, going way past the limit

Recommendations:

Check `totalRaised` against `maxContributionAmount` **at the end** of the function instead of at the start of the function.

Treasures: Acknowledged. This is a valid observation — however we have maxContributionPerPerson limit, which is set to 10K contribution token (USDC), so in the worst case (given maxContributionAmount of 3M), theoretical max of totalRaised will be (2,999,999 + 10,000) = 3,009,999 — which is okay as 0.3% more than maxContributionAmount is within our tolerance limit, given we have a fair proportional allocation mechanism when there is oversubscription (which ensures we always just take targetContributionAmount of 1M from contributors regardless of how much totalRaised is).

[M-2] `projectTaxPendingSwap` may exceed `type(uint128).max`

SEVERITY: Medium	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Low

Target

- FundToken.sol#L529-L559

Description:

In FundToken, `projectTaxPendingSwap` is of type uint128, assuming that `projectTaxPendingSwap` is always less than `type(uint128).max`.

```
uint128 public projectTaxPendingSwap;
...
    if (isLiquidityPool(to_) && projectSellTaxBasisPoints > 0) {
        if (projectSellTaxBasisPoints > 0) {
            uint256 projectTax = ((sentAmount_ * projectSellTaxBasisPoints)
    / BP_DENOM);
            projectTaxPendingSwap += uint128(projectTax);
            tax_ += projectTax;
        }
    }
    // on buy
    else if (isLiquidityPool(from_) && projectBuyTaxBasisPoints > 0) {
        if (projectBuyTaxBasisPoints > 0) {
            uint256 projectTax = ((sentAmount_ * projectBuyTaxBasisPoints)
    / BP_DENOM);
            projectTaxPendingSwap += uint128(projectTax);
            tax_ += projectTax;
        }
    }
```

However, when creating tokens, the protocol only verifies that `maxSupply / 1e18 ≤ type(uint128).max`, leaving the actual totalSupply potentially larger than `type(uint128).max`.

```
function _processSupplyParams(FundTokenSupplyParameters memory
    supplyParameters_) internal {
```

```
    if (
        supplyParameters_.maxSupply
            ≠ (supplyParameters_.lpSupply
    + supplyParameters_.fundVaultSupply
                    + supplyParameters_.privateContributionSupply
    + supplyParameters_.publicContributionSupply)
    ) {
        revert SupplyTotalMismatch();
    }

    if (supplyParameters_.maxSupply > type(uint128).max) {
        revert MaxSupplyTooHigh();
    }
...
    _processSupplyParams(supplyParams);

    uint256 lpSupply = supplyParams.lpSupply * (10 ** decimals());
    uint256 fundVaultSupply = supplyParams.fundVaultSupply
    * (10 ** decimals());
    uint256 privateContributionSupply
    = supplyParams.privateContributionSupply * (10 ** decimals());
    uint256 publicContributionSupply = supplyParams.publicContributionSupply
    * (10 ** decimals());
```

Later, when processing tax, `projectTaxPendingSwap` could exceed `type(uint128).max`. This causes an overflow, which prevents tokens from being sold or bought.

```
projectTaxPendingSwap += uint128(projectTax);
```

Recommendations:

```
if (
    params.maxSupply
            ≠ (params.lpSupply + params.fundVaultSupply
    + params.privateContributionSupply
            + params.publicContributionSupply) || params.maxSupply > type(
                uint128).max
            + params.publicContributionSupply) || params.maxSupply > type(
                uint128).max / 1e18
) {
    revert GenesisInvalidFundTokenSupplyParams();
}
...
```

```
        if (supplyParameters_.maxSupply > type(uint128).max) {
        if (supplyParameters_.maxSupply > type(uint128).max / 1e18) {
    revert MaxSupplyTooHigh();
}
```

Treasures: Resolved with PR 14.

Zenith: Verified.

[M-3] Insufficient slippage and deadline checks in `_swapTax()`

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target

- FundToken.sol#L689-L699

Description:

The Fund token contract swaps collected taxes on Uniswap after a certain threshold is reached:

```
function _swapTax(uint256 swapBalance_, address taxRecipient) internal {
    if (swapBalance_ > 0) {
        address[] memory path = new address[](2);
        path[0] = address(this);
        path[1] = pairToken;


    _uniswapRouter.swapExactTokensForTokensSupportingFeeOnTransferTokens(
            swapBalance_, 0, path, taxRecipient, block.timestamp + 600
        );
    }
}
```

There are two main issues with the current approach:

+ `amountOutMin` is hardcoded to 0, which exposes the contract to sandwich attacks. An attacker could manipulate the price to extract value from the swap, effectively stealing part of the collected taxes.

+ using `block.timestamp` directly to calculate the deadline is ineffective. Since `block.timestamp` reflects the time at the moment of swap execution, it does not provide meaningful protection against timing-related risks.

Consider the following scenarios.

Alice sells a large amount of FundToken, causing the FundToken price to drop. Alice transfers FundToken to Bob, which triggers autoSwap, and a large amount of FundToken (1% of totalSupply) are exchanged for pairToken at the low price, and the protocol receives

a lower amount of pairToken than expected. Alice swaps low-priced FundToken with pairToken to arbitrage.

Recommendations:

One solution is to specify both amountOutMin and deadline externally.

Treasures: Resolved with PR 15. We are addressing this by setting the absolute minimum swapThresholdBasisPoints, i.e. 1 basis points, or 0.01% of total supply (2.4M) — which amounts to 240 tokens, to only be autoswapped each time, which would give very little arbitrage gain to any attacker.

Zenith: Verified.

4.3 Low Risk

A total of 3 low risk findings were identified.

[L-1] End time cannot be adjusted once start time starts

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- Genesis.sol

Description:

In the genesis contract, the end time cannot be adjusted once the start time starts.

This is because the end time can be adjusted in the `resetTime` function, however that function has the `whenNotStarted` modifier.

```
function resetTime(uint256 newStartTime, uint256 newEndTime)
        external
        onlyOwner
        nonReentrant
        whenNotCancelled
        whenNotFailed
>       whenNotStarted
        .....
```

Recommendations:

Allow owner a more flexible option to push end time back even after the start time has begun.

Treasures: Acknowledged. This is a business design to not allow end time change once genesis has started, also we have a mechanism to extend genesis via resetMaxContributionAmount if needed.

[L-2] `lpSupply` and `contributionTokensForLpPool` cannot be 0

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target

- Genesis.sol#L172-L200

Description:

When `lpSupply` or `contributionTokensForLpPool` is 0, `addInitialLiquidity()` will fail to provide initial liquidity, resulting in tokens not being successfully launched.

```
function _addInitialLiquidity(address lpOwner) internal {
    // Funded date is the date of first funding. We can only add initial
    liquidity once. If this date is set,
    // we cannot proceed
    if (fundedDate ≠ 0) {
        revert InitialLiquidityAlreadyAdded();
    }

    fundedDate = uint32(block.timestamp);

    // Can only do this if this contract holds tokens:
    if (balanceOf(address(this)) = 0) {
        revert NoTokenForLiquidityPair();
    }

    // Approve the uniswap router for an infinite amount (max uint256)
    // This means that we don't need to worry about later incremental
    // approvals on tax swaps, as the uniswap router allowance will never
    // be decreased (see code in decreaseAllowance for reference)
    _approve(address(this), address(_uniswapRouter), type(uint256).max);
    IERC20(pairToken).approve(address(_uniswapRouter), type(uint256).max);
    // Add the liquidity:

    address pairAddr
    = IUniswapV2Factory(_uniswapRouter.factory()).getPair(address(this),
    pairToken);
```

```
    uint256 amountA = balanceOf(address(this));
    uint256 amountB = IERC20(pairToken).balanceOf(address(this));

    // This, effectively, transfer without applying taxes.
    // == transfer(pairAddr, amountA)
    super._update(address(this), pairAddr, amountA);

    IERC20(pairToken).safeTransfer(pairAddr, amountB);

    uint256 lpTokens = IUniswapV2Pair(pairAddr).mint(address(this));
```

Recommendations:

It is recommended to check that `lpSupply` and `contributionTokensForLpPool` are not 0.

Treasures: Resolved with PR 12.

Zenith: Verified.

[L-3] The actual swapped tax amount will be 1 less than the swapBalance

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: High

Target

- FundToken.sol#L567-L578

Description:

During autoSwap, the contract splits the `swapBalance` into `projectTreasurySwapBalance` and `fundVaultSwapBalance`. Since it's always calculated based on ShareBasisPoint, rounding will result in the actual swapped tax amount will be 1 less than the swapBalance.

```
(uint256 projectTreasurySwapBalance, uint256 fundVaultSwapBalance)
    = _splitTax(swapBalance);
...
function _splitTax(uint256 amountToSplit)
    internal
    view
    returns (uint256 projectTreasuryShareAmount_,
    uint256 fundVaultShareAmount_)
{
    // Not rechecking if tax enabled here, since this should only be called
    in autoswap,
    // which already does the checks.
    projectTreasuryShareAmount_ = ((amountToSplit
    * projectTreasuryTaxShareBasisPoint) / BP_DENOM);
    fundVaultShareAmount_ = ((amountToSplit * fundVaultTaxShareBasisPoint)
    / BP_DENOM);

    return (projectTreasuryShareAmount_, fundVaultShareAmount_);
}
```

Recommendations:

```
function _splitTax(uint256 amountToSplit)
    internal
    view
    returns (uint256 projectTreasuryShareAmount_,
    uint256 fundVaultShareAmount_)
{
    // Not rechecking if tax enabled here, since this should only be called
    in autoswap,
    // which already does the checks.
    projectTreasuryShareAmount_ = ((amountToSplit
    * projectTreasuryTaxShareBasisPoint) / BP_DENOM);

    fundVaultShareAmount_ = ((amountToSplit * fundVaultTaxShareBasisPoint)
            / BP_DENOM);
    fundVaultShareAmount_ = amountToSplit - projectTreasuryShareAmount_;

    return (projectTreasuryShareAmount_, fundVaultShareAmount_);
}
```

Treasures: Resolved with PR 13.

Zenith: Verified.

4.4 Informational

A total of 1 informational findings were identified.

[I-1] `cancelGenesis` has a redundant modifier

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target

- Genesis.sol

Description:

In `cancelGenesis`:

```
function cancelGenesis()
    external
    onlyOwner
    nonReentrant
    whenNotCancelled
    whenNotFailed
    whenNotStarted
    whenNotEnded
{
    isCancelled = true;
    emit GenesisCancelled();
}
```

Both the modifiers `whenNotStarted` and `whenNotEnded` is called.

However, if `whenNotStarted` is true then `whenNotEnded` is always true as we can see that throughout the code, whenever the start and end times are modified, `_validateTime` is always called.

And `_validateTime` ensures that `startTime` is **less** than `endTime`.

Recommendations:

Remove the redundant `whenNotEnded` call in this function to save gas.

Treasures: Resolved with PR 11.

Zenith: Verified.